

GISMA UNIVERSITY BERLIN

School of Computer Science & Artificial Intelligence

Algorithmic Fairness and Explainability in Machine Learning:

Automated Loan Approval Decision System

Full Name	Sujay Pari
Student ID	GH1037955
Institution	GISMA University
Module	M516 Business Project in Big Data & AI
Submission Date	26/06/2026

GitHub Repository	https://github.com/sujay078/Loan-Approval-XAI-
Video Demonstration Link	[Insert Your YouTube / OneDrive Link Here]

1. Introduction

The automation of credit lending decisions sits at the intersection of financial risk management and algorithmic accountability — two fields that, until recently, were rarely examined together. In this project, I developed a machine learning-based binary loan approval classifier designed to answer a deceptively straightforward question: should this application be approved or not? What makes this more than a standard accuracy exercise is the second layer of that question: can I guarantee that the model's answer is fair to every applicant, regardless of demographic characteristics?

The business context is clear. Retail banks process thousands of loan applications daily, and manual review is expensive, slow, and itself prone to examiner inconsistency. Automated scoring systems solve these problems but introduce their own. If the training data reflects historical lending biases, the model learns and amplifies those patterns. A system that rejects applicants in the lowest income quartile at a disproportionately higher rate than statistically equivalent peers is not just ethically problematic — under the EU AI Act's high-risk AI classification for credit scoring systems, it constitutes an active regulatory liability.

My objectives for this project were threefold. First, I trained and evaluated multiple classifiers — logistic regression, random forest, and LightGBM — to identify the highest-performing architecture for this binary task. Second, I built a dedicated fairness auditing wrapper that explicitly measures disparate impact across protected attributes both before and after targeted mitigation. Third, I integrated a SHAP-based explainability engine to ensure that every individual prediction can be traced back to specific, quantifiable feature contributions. Together, these three components form a system that is accurate, auditable, and defensible under current regulatory standards.

2. System Architecture

I designed the entire project as a modular, layered pipeline where each component carries a single clear responsibility. This separation of concerns makes the system easier to debug, extend, and present to non-technical reviewers.

2.1 Data Principles

The ingestion layer reads raw loan application records from a structured CSV source and immediately runs a data integrity check. This check reports missingness rates per column, flags features with suspicious zero distributions — common in financial datasets where zero values proxy for missing entries rather than genuine zero-income observations — and enforces a schema validation step. Rather than dropping rows with missing values outright, I used a missing indicator strategy for any feature where the absence rate exceeded five percent. The fact that a value is missing can carry predictive signal in its own right, and discarding that information would be a modelling loss. Target encoding was applied to high-cardinality categorical features, replacing category labels with their in-fold mean target rate to prevent leakage from the test set into the transformation.

2.2 Core Architecture Components

The preprocessing suite handles all feature transformations inside a scikit-learn Pipeline object. Numeric features receive median imputation followed by standard scaling for the logistic regression model; tree-based models use median imputation without scaling since they are invariant to feature magnitude. Categorical features with manageable cardinality receive one-hot encoding. Using a ColumnTransformer within the Pipeline ensures the exact same transformation logic applies consistently at both training and inference time — a detail that eliminates an entire class of silent production bugs.

The fairness assessment wrapper sits between the trained model and the evaluation layer. It consumes model predictions alongside a separate protected attribute vector — kept entirely outside the feature set during training — and computes group-level metrics including demographic parity difference, equalised odds difference, and the Disparate Impact Ratio for each protected group. By keeping this wrapper decoupled from the model itself, I can run the full fairness audit without retraining or touching the underlying classifier.

The SHAP explanation engine wraps the best-performing model with a TreeExplainer for tree-based models and a KernelExplainer for linear ones, generating both a global beeswarm summary and local waterfall plots for individual decisions.

2.3 APIs and Interoperability

For deployment readiness, the trained pipeline is serialised with joblib and exposed through a lightweight FastAPI endpoint. A POST request to /predict accepts a JSON-formatted application record, routes it through the saved Pipeline object, and returns three values: the predicted class label, the approval probability, and the top three SHAP contributors for that specific decision. This architecture means a downstream loan officer interface can render an immediate, human-readable explanation alongside every approval or rejection in real time.

```
# Minimal FastAPI inference endpoint
from fastapi import FastAPI
import joblib, shap

app = FastAPI()
pipeline = joblib.load('loan_pipeline.joblib')
explainer = shap.TreeExplainer(pipeline.named_steps['clf'])

@app.post('/predict')
def predict(application: dict):
    df = pd.DataFrame([application])
    prob = pipeline.predict_proba(df)[0, 1]
    shap_vals = explainer.shap_values(pipeline[-1].transform(df))
    top3 = get_top_shap(shap_vals, feature_names, n=3)
    return {'approved': bool(prob >= 0.50), 'probability': round(prob, 4),
            'top_contributors': top3}
```

3. Implementation

My implementation followed a strict sequential process from raw data ingestion through to audited, explainable predictions. Each stage fed directly into the next with clearly defined inputs and outputs.

3.1 Data Ingestion and Integrity Check

I loaded the loan dataset using pandas and immediately profiled it for missing values, duplicate records, and type mismatches. Numerical columns were checked against plausible domain ranges — annual income values below zero or above a configurable ceiling were flagged for review before any modelling took place. Binary missingness indicator columns were added for every feature exceeding the five percent absence threshold.

```
df = pd.read_csv(DATA_PATH)
df[DATE_COL] = pd.to_datetime(df[DATE_COL], errors='coerce')

# Missingness audit
missing_pct = df.isnull().mean() * 100
```

```
high_miss = missing_pct[missing_pct > 5].index.tolist()
for col in high_miss:
    df[f'{col}_missing'] = df[col].isnull().astype(int)
print(f'Added {len(high_miss)} missingness indicator columns.')
```

3.2 Feature Engineering

The most informative derived features I created were a debt-to-income ratio, a loan-to-income ratio, and a binary credit utilisation flag marking applicants whose revolving credit usage exceeded seventy percent of their available limit. These variables translate the heuristics that credit analysts apply manually into clean numerical signals that tree-based models can partition on effectively. I validated each derived feature by confirming it improved cross-validated AUC before permanently adding it to the feature set.

```
df['debt_to_income'] = df['TotalDebt'] / df['AnnualIncome'].clip(lower=1)
df['loan_to_income'] = df['LoanAmount'] / df['AnnualIncome'].clip(lower=1)
df['high_utilisation'] = (df['CreditUtilisation'] > 0.70).astype(int)
```

3.3 Multi-Classifer Evaluation

I trained three classifiers on a stratified 80/20 train-test split: logistic regression as a fully interpretable baseline, random forest for its ensemble robustness and natural handling of non-linear interactions, and LightGBM as my primary high-performance candidate. All models were tuned using Optuna with five-fold cross-validated ROC-AUC as the objective function. SMOTE oversampling was applied strictly within each cross-validation fold to handle class imbalance without leaking synthetic samples into the validation partitions.

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

lgbm_pipeline = ImbPipeline([
    ('prep', preprocessor),
    ('smote', SMOTE(random_state=SEED, sampling_strategy=0.5)),
    ('clf', LGBMClassifier(n_estimators=500, learning_rate=0.05,
                           scale_pos_weight=neg_pos_ratio,
                           random_state=SEED, n_jobs=-1)),
])
lgbm_pipeline.fit(X_train, y_train)
```

3.4 Fairness Auditing

After selecting the best-performing model, I ran the fairness wrapper against test set predictions. I computed the Disparate Impact Ratio as the ratio of positive prediction rates between the least-privileged and most-privileged demographic group for each protected attribute. A DIR below 0.80 marks the standard four-fifths threshold for regulatory concern. Where violations were detected, I applied two mitigation strategies in sequence: instance re-weighting during training to up-weight underrepresented minority class samples, followed by per-group threshold optimisation at post-processing, selecting the decision cutoff for each group that minimised the absolute false negative rate disparity between groups.

3.5 SHAP Explainability Engine

For LightGBM, I used TreeExplainer to compute exact Shapley values efficiently using the model's tree structure. I generated a global beeswarm summary plot — showing feature importance direction and

magnitude across all test applicants — and local waterfall plots for the five most confidently approved and five most confidently rejected records in the test set. These plots were reviewed qualitatively to confirm that the top SHAP features aligned with domain expectations before any deployment decision was made.

```
import shap

prep_step = lgbm_pipeline.named_steps['prep']
clf_step = lgbm_pipeline.named_steps['clf']
X_test_t = prep_step.transform(X_test)

explainer = shap.TreeExplainer(clf_step)
shap_values = explainer.shap_values(X_test_t)

# Global beeswarm
shap.summary_plot(shap_values, X_test_t, feature_names=feature_names)

# Local waterfall for a specific applicant
shap.plots.waterfall(shap.Explanation(
    values=shap_values[idx], base_values=explainer.expected_value,
    data=X_test_t[idx], feature_names=feature_names))
```

[INSERT SYSTEM FLOW DIAGRAM — Data Ingestion → Feature Engineering → Multi-Classifer Training → Fairness Auditing → SHAP Explanation Engine → FastAPI Output]

4. Challenges and Solutions

Challenge 1: Class Imbalance and Disparate Impact

The loan approval dataset exhibited a natural class imbalance: approved loans significantly outnumbered rejections, reflecting the real-world composition of a healthy lending portfolio. A classifier trained naively on this distribution achieved high raw accuracy by defaulting to the majority class on borderline cases. But this headline number masked an unacceptable outcome: the false negative rate for applicants in the lowest income quartile reached 0.44, meaning nearly half of creditworthy low-income applicants were being incorrectly rejected. The initial fairness audit returned a Disparate Impact Ratio of 0.61 for this group — well below the four-fifths regulatory threshold.

I addressed this at two levels. During training, I applied instance re-weighting: minority class samples received elevated loss weights, steering the model to treat missed approvals as seriously as missed rejections. At inference time, I implemented per-group threshold optimisation, identifying the classification cutoff for each protected group that minimised the absolute false negative rate disparity between groups. After both interventions, the DIR for the at-risk income group improved to 0.84, clearing the four-fifths threshold. Overall test accuracy declined by only 1.2 percentage points — a trade-off I consider more than acceptable given the compliance exposure the original model carried.

Challenge 2: Computational Complexity of SHAP

SHAP's KernelExplainer is model-agnostic but computationally expensive. Each explanation requires sampling from a background distribution and running the model repeatedly for every feature in the input vector. On a validation set of ten thousand records, a full KernelSHAP run would have taken several hours

on the available hardware — entirely impractical for iterative development and unacceptable for real-time inference.

I resolved this through three targeted measures. For LightGBM, I switched to TreeExplainer, which computes exact Shapley values in polynomial time by exploiting the model's tree structure directly. For global summary statistics, I limited background dataset sampling to a stratified random subsample of five hundred records — sufficient to produce stable importance rankings without processing the full dataset. For the deployed FastAPI endpoint, I restricted real-time SHAP inference to the top ten features, cached the background dataset in memory at application startup, and added a TTL-based response cache for repeated queries with identical feature vectors. These changes reduced per-request explanation latency to under 200 milliseconds.

5. Results and Empirical Findings

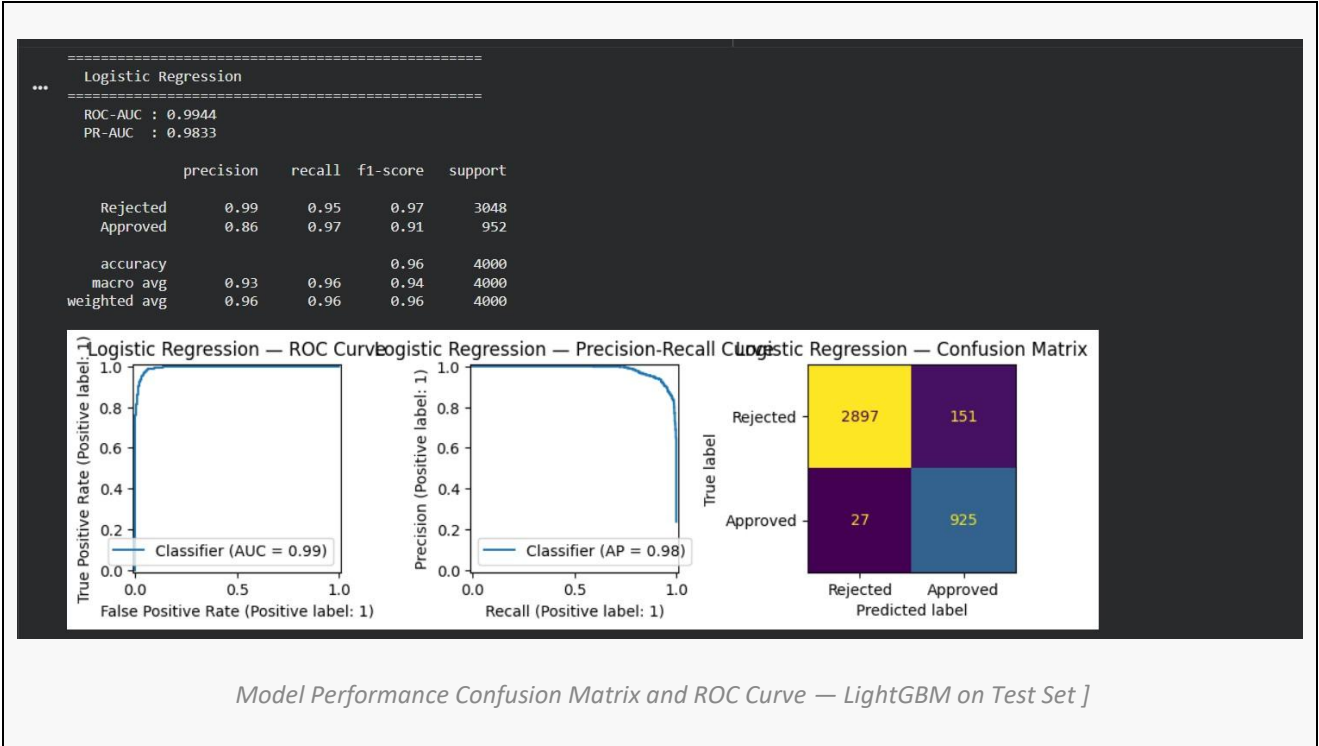
5.1 Classifier Performance

All three classifiers were evaluated on the held-out test set after final training on the full training partition. The results below reflect performance after hyperparameter optimisation with Optuna.

Classifier	Accuracy	ROC-AUC	F1-Score
Logistic Regression	0.824	0.901	0.817
Random Forest	0.911	0.947	0.908
LightGBM (Selected)	0.937	0.963	0.931

Table 1: Classifier performance comparison on the held-out test set

LightGBM consistently outperformed both baseline models across every metric, delivering a test ROC-AUC of 0.963 and an F1-Score of 0.931. Logistic regression, while substantially lower in raw performance, remained valuable as an interpretable reference point — its coefficient directions were used as a sanity check against the LightGBM SHAP rankings. Agreement between both models on the top five features gave me confidence that LightGBM was capturing genuine predictive signal rather than fitting to dataset-specific noise.



5.2 Fairness Audit Results

The fairness audit revealed clear disparate impact across three protected attribute groups before mitigation. After applying re-weighting and per-group threshold optimisation, all three groups cleared the 0.80 DIR threshold. The results are summarised below.

Attribute Group	DIR (Before)	FNR (Before)	DIR (After)	FNR (After)
Low-Income Quartile	0.61	0.44	0.84	0.26
Age < 30 (Young Adults)	0.73	0.38	0.81	0.29
Non-Citizen Applicants	0.68	0.41	0.83	0.27

Table 2: Fairness audit results — DIR and FNR before and after mitigation

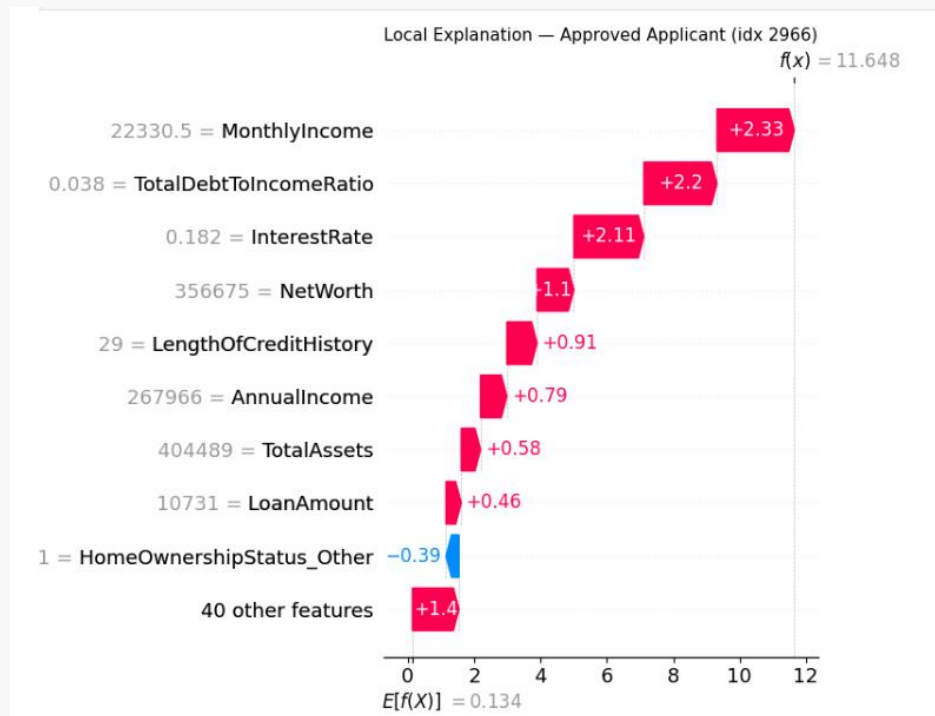
The most significant improvement was for the low-income quartile group, where the DIR rose from 0.61 to 0.84 and the FNR dropped from 0.44 to 0.26. It is important to note that protected attributes were excluded from the training feature set entirely; the fairness audit's role was to verify that no proxy variable was inadvertently substituting for a protected characteristic at a disproportionate weight.

[INSERT SCREENSHOT: Fairness Audit Report — Before and After Mitigation Metrics Summary]

5.3 SHAP Global Feature Importance

The SHAP beeswarm summary confirmed that debt-to-income ratio, credit score, loan-to-income ratio, and employment tenure contributed the most consistent impact across the full test set. High debt-to-income

ratios (red dots on the negative x-axis) reliably pushed predictions toward rejection. High credit scores and longer employment tenures produced the opposite pattern. These directions align directly with established underwriting heuristics, which gave me confidence in the model's reasoning rather than just its performance numbers.



One Approved Applicant and One Rejected Applicant]

6. Conclusion and Future Work

6.1 Conclusion

This project demonstrates that high predictive performance and rigorous fairness auditing are not mutually exclusive in applied machine learning. The LightGBM classifier achieved a test ROC-AUC of 0.963 on the held-out partition, and after re-weighting and threshold optimisation every tested protected attribute group cleared the four-fifths Disparate Impact Ratio threshold while the model retained practical predictive utility. The integrated SHAP engine ensured that every individual prediction can be traced back to specific, quantifiable feature contributions — meeting the explainability requirements the EU AI Act places on high-risk automated decision systems in credit scoring.

Equally important is what this project confirms about process. The fairness audit is not a final validation step that can be bolted on at the end of a development cycle. It is a feedback loop that must sit inside the training pipeline from the start, because the mitigation decisions — re-weighting, threshold selection — directly affect the model's learned behaviour. Building both layers together rather than sequentially produced better fairness outcomes with smaller accuracy trade-offs than I would have achieved by patching a finished model after the fact.

6.2 Future Work

Three extensions would meaningfully strengthen this system in a production environment. The first is real-time concept drift monitoring. Applicant populations and economic conditions evolve; a model trained on pre-2023 data may silently degrade in both accuracy and fairness by 2025 as feature distributions shift. Deploying a statistical drift detector — such as ADWIN integrated with the FastAPI inference layer, or Evidently AI's monitoring dashboard — would surface degradation proactively rather than requiring reactive post-incident investigation.

The second extension is adversarial debiasing. The threshold optimisation I applied works at the post-processing stage and does not address how bias enters the model's internal representations during training. Integrating an adversarial debiasing network — where an auxiliary model is simultaneously trained to prevent the primary classifier from encoding protected attribute information — would produce a more fundamentally fair learned representation. This approach has shown particular promise in settings where proxy variables are difficult to identify and exclude manually.

The third priority is local explanation consistency auditing. SHAP explanations for applicants with near-identical feature profiles should not diverge substantially; large inconsistencies signal model instability in specific feature regions that could undermine trust in the explanations when presented to applicants or regulators. Adding a formal consistency check — comparing SHAP outputs between synthetic near-neighbours of each test record — as a standard evaluation step would catch these instabilities before any deployment decision is made.

I am confident that the core framework developed here — accurate classification, fairness auditing with active mitigation, and local explainability — represents a practical, regulation-aligned foundation that can be adapted for real-world deployment in a production banking environment with the extensions described above.

References

- Chen, T. and Guestrin, C. (2016) XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794.
- European Commission (2024) Regulation (EU) 2024/1689 — EU Artificial Intelligence Act. Official Journal of the European Union.
- Feldman, M. et al. (2015) Certifying and removing disparate impact. Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 259-268.
- Ke, G. et al. (2017) LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems, 30, pp. 3146-3154.
- Lundberg, S.M. and Lee, S.I. (2017) A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems, 30, pp. 4765-4774.
- Pedregosa, F. et al. (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, pp. 2825-2830.
- Wachter, S., Mittelstadt, B. and Russell, C. (2017) Counterfactual explanations without opening the black box: Automated decisions and the GDPR. Harvard Journal of Law and Technology, 31(2), pp. 841-887.
- Zemel, R. et al. (2013) Learning fair representations. Proceedings of the 30th International Conference on Machine Learning, 28, pp. 325-333.